

CS 420

NFA $\Leftrightarrow$ DFA

Thursday, October 8, 2026

UMass Boston Computer Science

A *nondeterministic finite automaton*

is a mk-NFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_{\epsilon} \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.



A *finite automaton* is a mk-DFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

Announcements

- HW 4
 - Out: Tue 10/6
 - Due: Tue 10/13 12:30pm

A *nondeterministic finite automaton* is a mk-NFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_{\epsilon} \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.



A *finite automaton* is a mk-DFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

Previously

Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Want to prove:

THEOREM

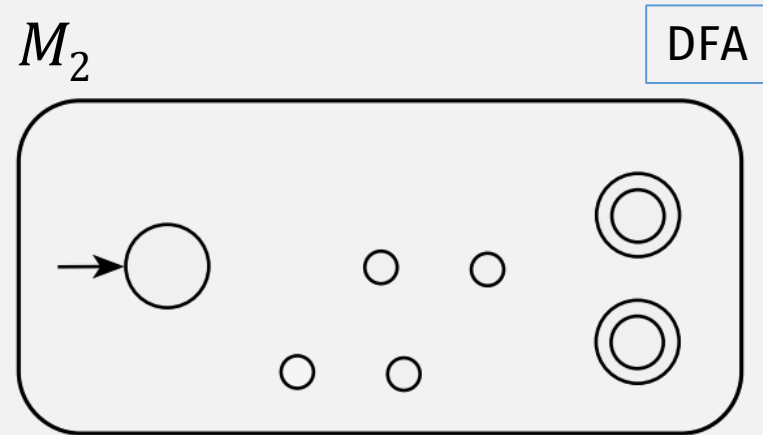
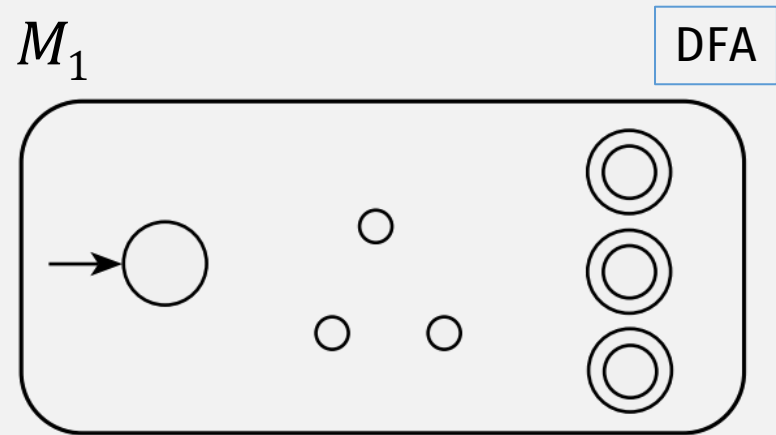
The class of regular languages is closed under the concatenation operation.

In other words, IF A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

Proof requires: Constructing new machine

- How does it know when to switch machines?
 - Can only read input once

Concatentation

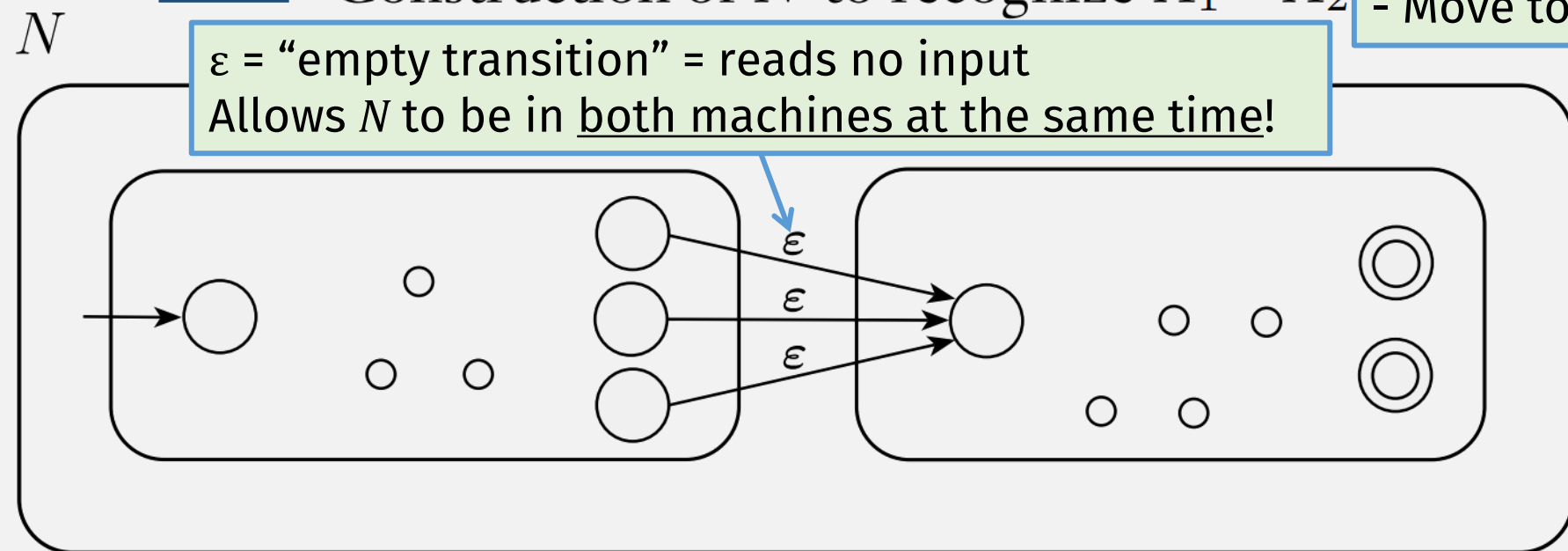


Have: recognize A_1 recognize A_2 .

Want: Construction of N to recognize $A_1 \circ A_2$

N is an **NFA**! It can:

- Keep checking 1st part with M_1
- and
- Move to M_2 to check 2nd part



NFA!

Define function $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$

$\text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2) = N$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

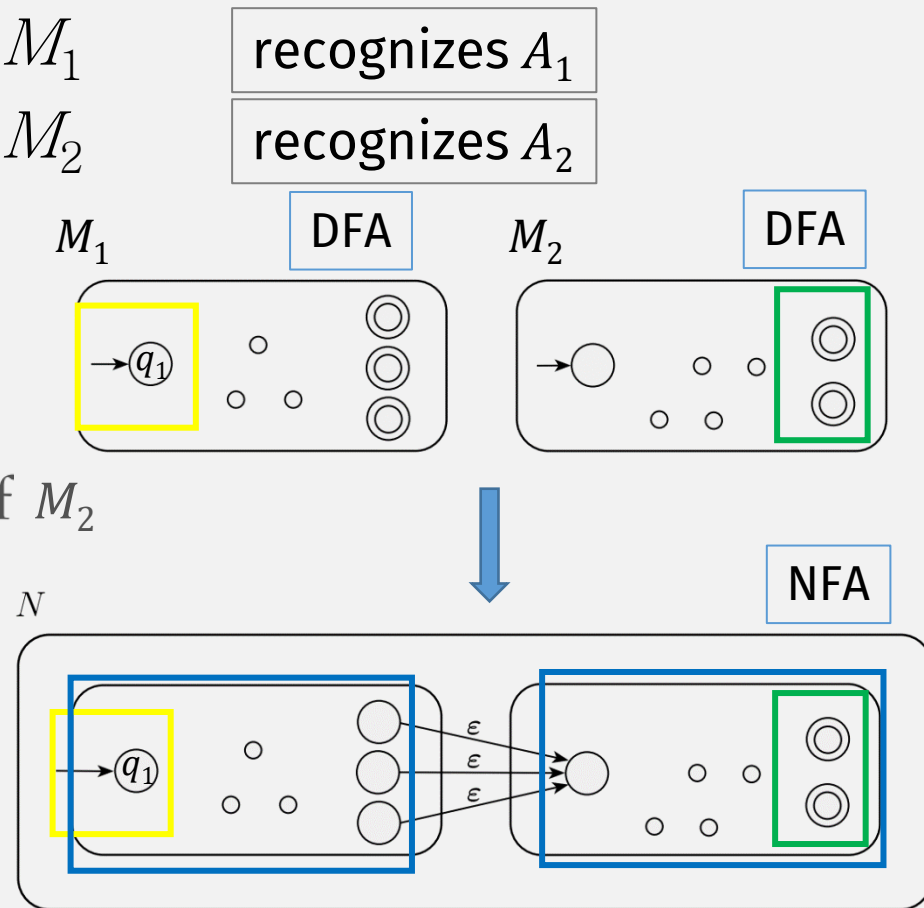
(Output) $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$ where

$$Q = Q_1 \cup Q_2$$

The state q_1 is the same as the start state of M_1

The accept states F_2 are the same as the accept states of M_2

$\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ where: for any $q \in Q$ and any $a \in \Sigma$



Define function $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$

$\text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2) = N$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

(Output) $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$ where

$$Q = Q_1 \cup Q_2$$

The state q_1 is the same as the start state of M_1

The accept states F_2 are the same as the accept states of M_2

$\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ where: for any $q \in Q$ and any $a \in \Sigma$

NFA def says δ must map every state and ϵ to set of states

$$\delta(q, a) = \begin{cases} \{\delta_1(q, a)\} & q \in Q_1 \text{ and } q \notin F_1 \\ \{\delta_1(q, a)\} & q \in F_1 \text{ and } a \neq \epsilon \\ ? & q \in F_1 \text{ and } a = \epsilon \\ \{\delta_2(q, a)\} & q \in Q_2 \end{cases}$$

And: $\delta(q, \epsilon) = \emptyset$, for $q \in Q, q \notin F_1$

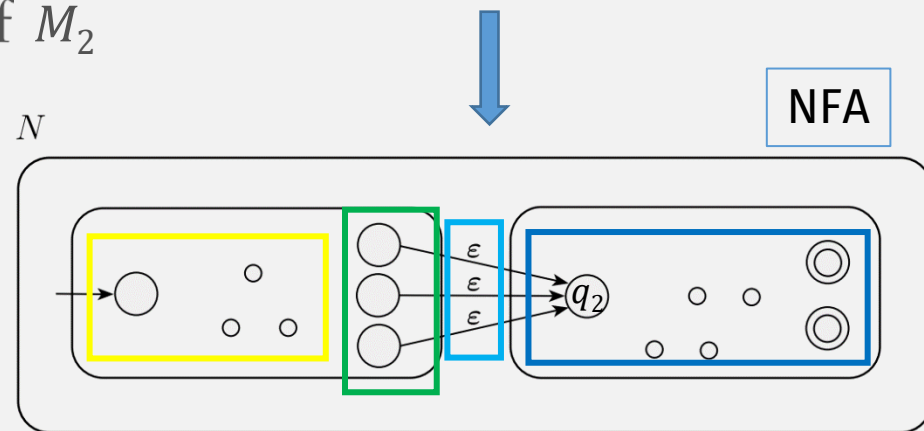
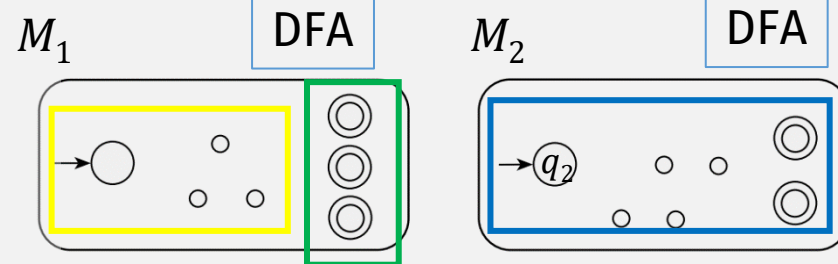
("piecewise" function definition)

IF A_1 IS-A **Regular Language**
AND
 A_2 IS-A **Regular Language**
THEN $A_1 \circ A_2$ IS-A **Regular Language**

So, is this true?

recognizes A_1

recognizes A_2



???

Previously

Regular Langs closed under Union?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA Machine** AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA Machine** AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A **DFA Machine**
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO ~~M_1~~ : **FIRST** Proof₃, **FIRST** Proof₄
6. BY ExTable₁ **COMPARING** $M, A_1 \cup A_2$
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

Regular Langs closed under Concat?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
 THEN $A_1 \circ A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2)$
 IS-A **NFA** Machine
6. M **recognizes** $A_1 \circ A_2$
7. $A_1 \circ A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
 TO Proof₁
4. APPLY DefRegLangCoro
 TO Proof₂
5. APPLY $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$
 TO FIRST Proof₃, FIRST Proof₄
6. BY ExTable₁ **COMPARING** $M, A_1 \circ A_2$
7. APPLY DefRegLang
 TO mk-AND-proof Proof₅, Proof₆

Regular Langs closed under Concat?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND M_1 recognizes A_1 DefRegLang
4. IF M IS-A **DFA** Machine AND M recognizes L
THEN L IS-A **Regular** Language
5. let $M = \text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2)$
IS-A **NFA** Machine Doesn't match!
6. M recognizes $A_1 \circ A_2$
7. $A_1 \circ A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro TO Proof₁
4. APPLY DefRegLangCoro TO Proof₂
5. APPLY $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$
TO FIRST Proof₃, FIRST Proof₄
6. BY ExTable₁ COMPARING $M, A_1 \circ A_2$
7. APPLY DefRegLang TO mk-AND-proof Proof₅, Proof₆

IF M IS-A **DFA** Machine AND M **recognizes** L
THEN L IS-A **Regular** Language

A DFA's Language

- For **DFA** $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ where
- M **accepts** w if $\hat{\delta}(q_0, w) \in F$
- M **recognizes** language $\{w \mid M \text{ accepts } w\}$

Definition: A DFA's language is a **regular language**

An NFA's Language?

IF N IS-A **NFA** Machine AND N recognizes L
THEN L IS-A ??? Language

- For **NFA** $N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F)$ where
 - Intersection ...
 - ... with accept states ...
- N *accepts* w if $\hat{\delta}(q_0, w) \cap F \neq \emptyset$
 - ... is not empty set
 - i.e., accept if final states contains at least one accept state
- Language of $N = \text{LangOf}(N) = \left\{ w \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset \right\}$

Q: What kind of languages do NFAs recognize?

Concatenation Closed for Reg Langs?

- Combining DFAs to recognize concatenation of languages ...
... produces an NFA
- So to prove concatenation is closed ...
... we must prove that NFAs also recognize regular languages.

Specifically, we must show:

NFAs recognize Regular languages too!

“IF AND ONLY IF” Statements

$$X \Leftrightarrow Y = \text{“}X \text{ IF AND ONLY IF } Y\text{”} = X \text{ IFF } Y = X \Leftrightarrow Y$$

Represents two statements:

Acceptable notation in this course

1. $\Rightarrow$ IF X THEN Y
 - “forward” direction

2. $\Leftarrow$ IF Y THEN X
 - “reverse” direction

How to Prove an “IFF” Statement

$$X \Leftrightarrow Y = \text{“}X \text{ IF AND ONLY IF } Y\text{”} = X \text{ IFF } Y = X \Leftrightarrow Y$$

Proof has two (IF-THEN proof) parts:

1. $\Rightarrow$ IF X THEN Y
 - “**forward**” direction
 - assume X , then use it to prove Y
2. $\Leftarrow$ IF Y THEN X
 - “**reverse**” direction
 - assume Y , then use it to prove X

Proving NFAs Recognize Regular Langs

L IS-A **Regular** Language RegLangIFFNFA

IF AND ONLY IF

(some) N IS-A **NFA** Machine AND N **recognizes** L

Need 2 proofs:

$\Rightarrow$ IF L IS-A **Regular** Language THEN N IS-A **NFA** Machine AND N **recognizes** L

Constructor?

$\Leftarrow$ IF N IS-A **NFA** Machine AND N **recognizes** L THEN L IS-A **Regular** Language

$\Rightarrow$ IF L IS-A **Regular** Language
THEN N IS-A NFA Machine
AND N **recognizes** L

Assume IF part ...

mk-IFTHEN-proof (with ASSUMPTION = proof of L IS-A **Regular** Language):

Statements

1. M IS-A DFA Machine AND
 M **recognizes** L
2. N
IS-A NFA Machine
3. N **recognizes** L
4. N IS-A NFA Machine AND
 N **recognizes** L

... prove THEN part

Justifications

1. **APPLY** DefRegLangCoro
TO ASSUMPTION

DefRegLangCoro

IF L IS-A **Regular** Language
THEN M IS-A **DFA** Machine AND M **recognizes** L

3. **BY** ExTab? **COMPARING** N, L

4. **mk-AND-proof** Proof_{2?} Proof_{3?}

$\Rightarrow$ IF L IS-A **Regular** Language
THEN N IS-A NFA Machine
AND N **recognizes** L

mk-IFTHEN-proof (with ASSUMPTION = proof of L IS-A **Regular** Language):

Statements

1. M IS-A DFA Machine AND
 M **recognizes** L
2. let N **????**
IS-A NFA Machine See HW4!
3. N **recognizes** L
4. N IS-A NFA Machine AND
 N **recognizes** L

Justifications

1. APPLY DefRegLangCoro
TO ASSUMPTION
2. APPLY $\text{CONVERT}_{\text{DFA-NFA}} : \text{DFA} \rightarrow \text{NFA}$
TO FIRST Proof₁
3. BY ExTab? **COMPARING** N, L
4. mk-AND-proof Proof₂? Proof₃?

“Proving” Machine Recognizes Language

Let:

$$\text{NFA } N = \text{CONVERT}_{\text{DFA-NFA}} M$$

let $s =$	$s \in L$	run M on s	run N on s ?	run N on s Justification
$w \in L$	Yes	$q_0, \dots, q_n$ where $q_n \in F$	Accept	See HW4!
$w' \notin L$	No	$q_0, \dots, q_m$ where $q_m \notin F$	Reject	
...				

Do we know concrete N and L ?

No! need justification

Proving NFAs Recognize Regular Langs

L IS-A **Regular** Language

RegLangIFFNFA

IF AND ONLY IF

(some) N IS-A **NFA Machine** AND N **recognizes** L

Need 2 proofs:

☑ $\Rightarrow$ IF L IS-A **Regular** Language THEN N IS-A **NFA Machine** AND N **recognizes** L

$\Leftarrow$ IF N IS-A **NFA Machine** AND N **recognizes** L THEN L IS-A **Regular** Language

Constructor?

⇐ IF N IS-A NFA Machine

AND N recognizes L

THEN L IS-A **Regular Language**

Assume IF part ...

mk-IFTHEN-proof (with ASSUMPTION = proof of N IS-A NFA Machine AND N recognizes L):

Statements

1. let $M =$
IS-A DFA Machine ???
2. M recognizes L
3. L IS-A **Regular Language**

... prove THEN part

Justifications

2. BY ExTab? COMPARING M, L
3. APPLY DefRegLang
TO mk-AND-proof Proof₁? Proof₂?

DefRegLang

IF M IS-A **DFA Machine** AND M recognizes L
THEN L IS-A **Regular Language**

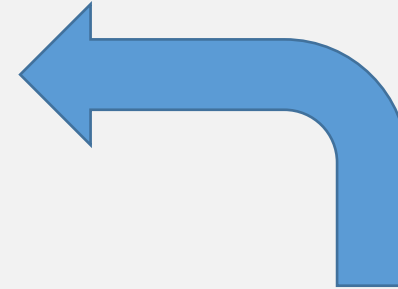
How to convert NFA→DFA?

A *finite automaton* is a mk-DFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

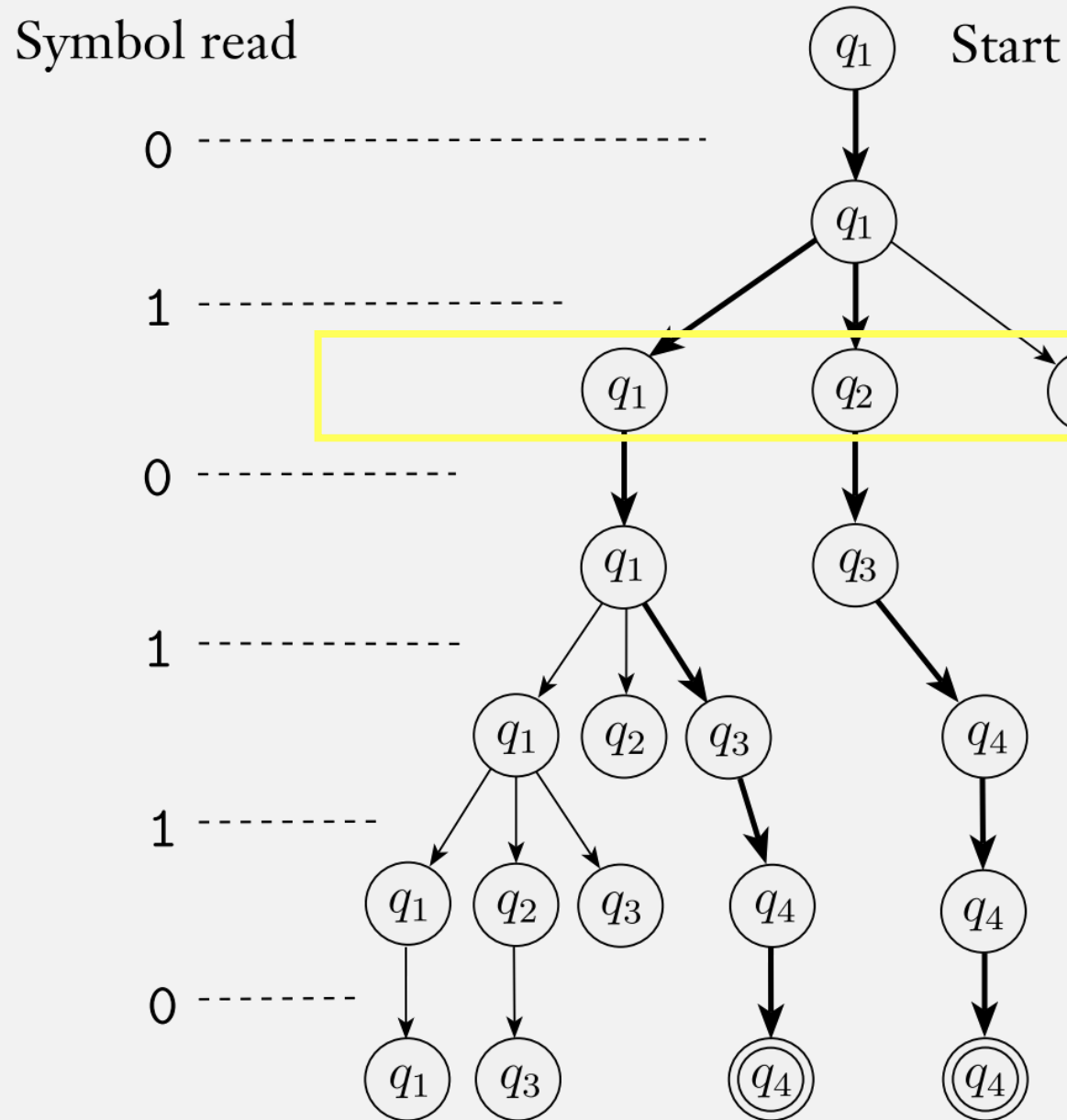
Proof idea:

Let each “state” of the DFA
= set of states in the NFA



A *nondeterministic finite automaton* is a mk-NFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.



NFA computation can be in multiple states

DFA computation can only be in one state

So encode:
a set of NFA states
as one DFA state

This is similar to the proof strategy from
“Closure of union” where:
a state = a pair of states

Convert NFA→DFA, Formally

$\text{CONVERT}_{\text{NFA-DFA}} : \text{NFA} \rightarrow \text{DFA}$

$\text{CONVERT}_{\text{NFA-DFA}} N = M$ where

$(Q_{\text{NFA}}, \Sigma, \delta_{\text{NFA}}, q_{0\text{NFA}}, F_{\text{NFA}}) = \text{get-NFA-parts } N$

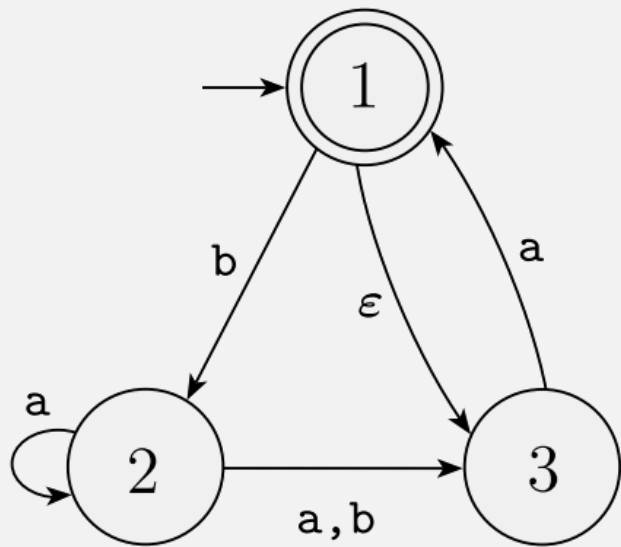
$M = \text{mk-DFA } (Q_{\text{DFA}}, \Sigma, \delta_{\text{DFA}}, q_{0\text{DFA}}, F_{\text{DFA}})$ where

NFA N with states Q will be converted to ...

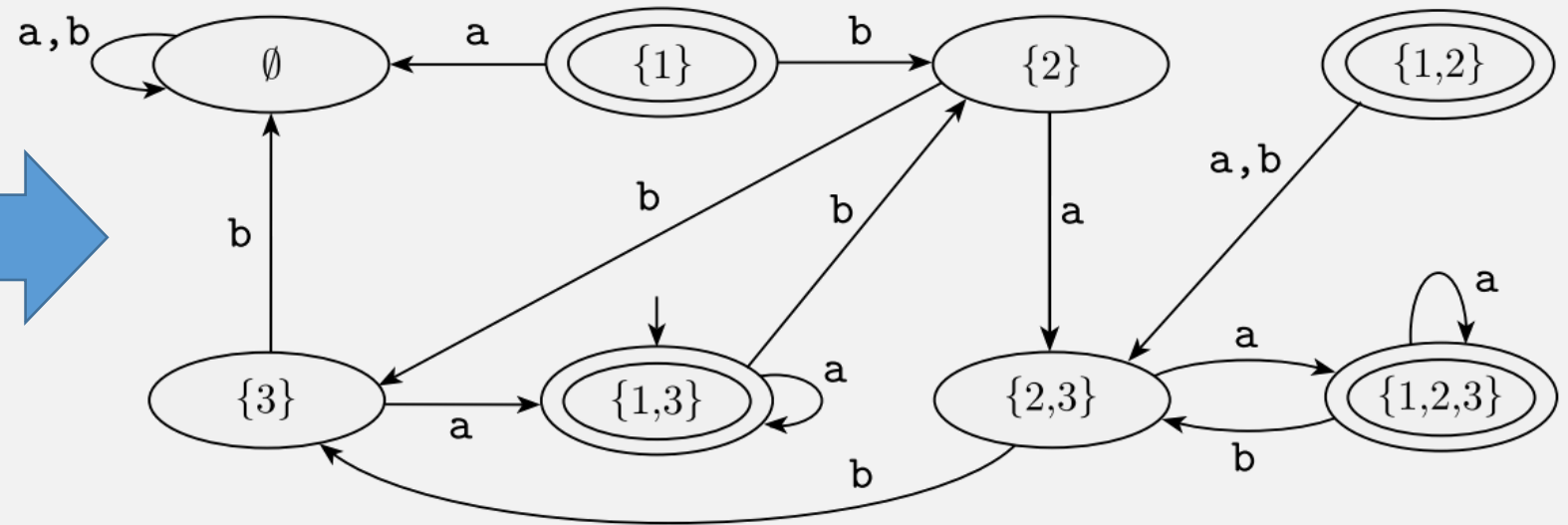
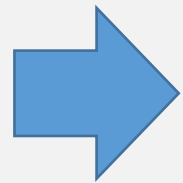
... an equivalent DFA M with states $Q' = \mathcal{P}(Q)$ (power set of Q)

Example:

- Let NFA $N_4 = \text{mk-NFA } (Q, \Sigma, \delta, q_0, F)$
- An equivalent DFA D has states $= \mathcal{P}(Q)$ (power set of Q)



The NFA N_4



A DFA D that is equivalent to the NFA N_4

No empty transitions

$\text{CONVERT}_{\text{NFA-DFA}} : \text{NFA} \rightarrow \text{DFA}$

$\text{CONVERT}_{\text{NFA-DFA}} N = M$ where

$(Q_{\text{NFA}}, \Sigma, \delta_{\text{NFA}}, q_{0\text{NFA}}, F_{\text{NFA}}) = \text{get-NFA-parts } N$

$M = \text{mk-DFA } (Q_{\text{DFA}}, \Sigma, \delta_{\text{DFA}}, q_{0\text{DFA}}, F_{\text{DFA}})$ where

$$Q_{\text{DFA}} = \mathcal{P}(Q_{\text{NFA}})$$

A DFA state = a set of NFA states

$qs = \text{DFA state} = \text{set of NFA states}$

For $qs \in Q_{\text{DFA}}$ and $a \in \Sigma$

$$\delta_{\text{DFA}}(qs, a) = \bigcup_{q \in qs} \delta_{\text{NFA}}(q, a)$$

A DFA step = an NFA step for all states in the set

$$q_{0\text{DFA}} = \{q_{0\text{NFA}}\}$$

$$F_{\text{DFA}} = \{ qs \in Q_{\text{DFA}} \mid qs \text{ contains accept state of } N \}$$

Flashback: Adding Empty Transitions

- Define the set $\varepsilon\text{-REACHABLE}(q)$
 - ... to be all states reachable from q via zero or more empty transitions

(Defined recursively)

- Base case: $q \in \varepsilon\text{-REACHABLE}(q)$

- Recursive case:

A state is in the reachable set if ...

$$\varepsilon\text{-REACHABLE}(q) = \{r \mid p \in \varepsilon\text{-REACHABLE}(q) \text{ and } r \in \delta(p, \varepsilon)\}$$

... there is an empty transition to it from another state in the reachable set

With empty transitions

$\text{CONVERT}_{\text{NFA-DFA}} : \text{NFA} \rightarrow \text{DFA}$

$\text{CONVERT}_{\text{NFA-DFA}} N = M$ where

$(Q_{\text{NFA}}, \Sigma, \delta_{\text{NFA}}, q_{0\text{NFA}}, F_{\text{NFA}}) = \text{get-NFA-parts } N$

$M = \text{mk-DFA } (Q_{\text{DFA}}, \Sigma, \delta_{\text{DFA}}, q_{0\text{DFA}}, F_{\text{DFA}})$ where

Almost the same, except ...

$$Q_{\text{DFA}} = \mathcal{P}(Q_{\text{NFA}})$$

For $qs \in Q_{\text{DFA}}$ and $a \in \Sigma$

$$\delta_{\text{DFA}}(qs, a) = \bigcup_{q \in qs} \delta_{\text{NFA}}(q, a) \rightarrow \bigcup_{s \in S} \varepsilon\text{-REACHABLE}(s)$$

$$q_{0\text{DFA}} = \{q_{0\text{NFA}}\} \varepsilon\text{-REACHABLE}(q_{0\text{NFA}})$$

$$F_{\text{DFA}} = \{ qs \in Q_{\text{DFA}} \mid qs \text{ contains accept state of } N \}$$

⇐ IF N IS-A NFA Machine
AND N recognizes L
THEN L IS-A **Regular** Language

mk-IFTHEN-proof (with ASSUMPTION = proof of N IS-A NFA Machine AND N recognizes L):

Statements

1. let $M =$
IS-A DFA Machine
2. M recognizes L
3. L IS-A **Regular** Language

Justifications

1. APPLY $\text{CONVERT}_{\text{NFA-DFA}} : \text{NFA} \rightarrow \text{DFA}$
TO FIRST ASSUMPTION
2. BY ExTab? **COMPARING** M, L
3. APPLY DefRegLang **TODO!** (Do on your own!)
TO mk-AND-proof Proof₁? Proof₂?

Proving NFAs Recognize Regular Langs

L IS-A **Regular** Language

RegLangIFFNFA

IF AND ONLY IF

(some) N IS-A NFA Machine AND N **recognizes** L

Need 2 proofs:

☑ $\Rightarrow$ IF L IS-A **Regular** Language THEN N IS-A NFA Machine AND N **recognizes** L

☑ $\Leftarrow$ IF N IS-A NFA Machine AND N **recognizes** L THEN L IS-A **Regular** Language



Define function $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$

$\text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2) = N$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-DFA-parts } M_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-DFA-parts } M_2$

(Output) $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$ where

$$Q = Q_1 \cup Q_2$$

The state q_1 is the same as the start state of M_1

The accept states F_2 are the same as the accept states of M_2

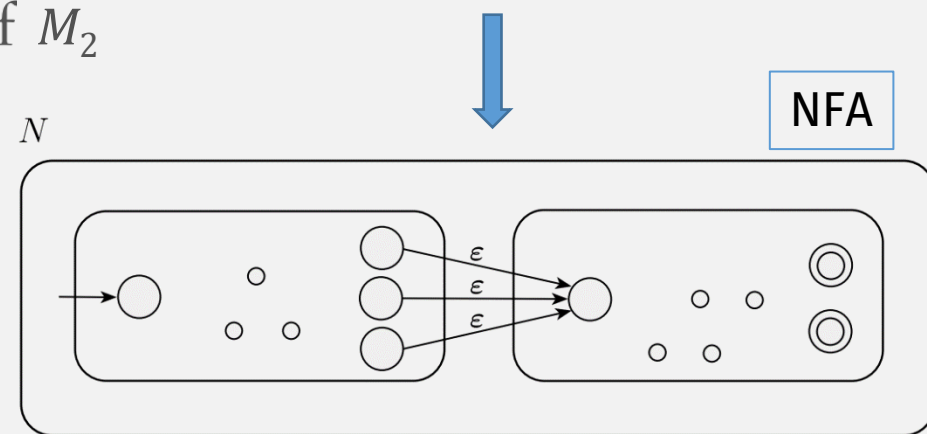
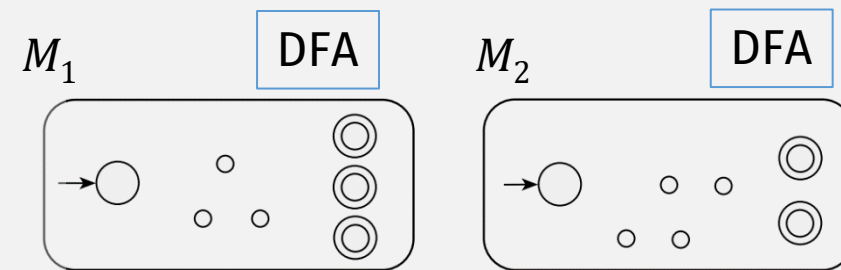
$\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ where: for any $q \in Q$ and any $a \in \Sigma$

$$\delta(q, a) = \begin{cases} \{\delta_1(q, a)\} & q \in Q_1 \text{ and } q \notin F_1 \\ \{\delta_1(q, a)\} & q \in F_1 \text{ and } a \neq \epsilon \\ \{q_2\} & q \in F_1 \text{ and } a = \epsilon \\ \{\delta_2(q, a)\} & q \in Q_2. \end{cases}$$

And: $\delta(q, \epsilon) = \emptyset$, for $q \in Q, q \notin F_1$

IF A_1 IS-A **Regular Language**
AND
 A_2 IS-A **Regular Language**
THEN $A_1 \circ A_2$ IS-A **Regular Language**

So, is this true?



???

Regular Langs closed under Concat?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND M_1 recognizes A_1 DefRegLang
4. IF M IS-A **DFA** Machine AND M recognizes L
THEN L IS-A **Regular** Language
5. let $N = \text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2)$
IS-A **NFA** Machine Doesn't match!
6. N recognizes $A_1 \circ A_2$
7. $A_1 \circ A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro TO Proof₁
4. APPLY DefRegLangCoro TO Proof₂
5. APPLY $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$
TO FIRST Proof₃, FIRST Proof₄
6. BY ExTable₁ COMPARING $N, A_1 \circ A_2$
7. APPLY DefRegLang TO mk-AND-proof Proof₅, Proof₆

Regular Langs closed under Concat?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A DFA Machine AND
 M_1 recognizes A_1
4. M_2 IS-A DFA Machine AND
 M_2 recognizes A_2
5. let $N = \text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2)$
IS-A **NFA** Machine
6. N recognizes $A_1 \circ A_2$
7. $A_1 \circ A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. L IS-A **Regular** Language
IF AND ONLY IF
 N IS-A NFA Machine AND N recognizes L
RegLangIFFNFA
4. TO Proof₂
5. APPLY $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$
TO FIRST Proof₃, FIRST Proof₄
6. BY ExTable₁ COMPARING $N, A_1 \circ A_2$
7. APPLY DefRegL RegLangIFFNFA $\Leftarrow$
TO mk-AND-proof Proof₅, Proof₆

When using an IFF with APPLY,
must specify a direction!

Finally done with this proof!

Regular Langs closed under Concat?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $N = \text{CONCAT}_{\text{DFA-NFA}}(M_1, M_2)$
IS-A **NFA** Machine
6. N **recognizes** $A_1 \circ A_2$
7. $A_1 \circ A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{CONCAT}_{\text{DFA-NFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{NFA}$
TO FIRST Proof₃, FIRST Proof₄
6. BY ExTable₁ **COMPARING** $N, A_1 \circ A_2$
7. APPLY $\text{RegLangIFFNFA} \Leftarrow$
TO mk-AND-proof Proof₅, Proof₆

New possible proof strategy!

Use **NFAs** Only

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. N_1 IS-A **NFA** Machine AND
 N_1 **recognizes** A_1
4. N_2 IS-A **NFA** Machine AND
 N_2 **recognizes** A_2
5. let $N = \text{CONCAT}_{\text{NFA}}(N_1, N_2)$ **???**
IS-A **NFA** Machine
6. N **recognizes** $A_1 \circ A_2$
7. $A_1 \circ A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY **RegLangIFFNFA** $\Rightarrow$
TO Proof₁
4. APPLY **RegLangIFFNFA** $\Rightarrow$
TO Proof₂
5. APPLY **CONCAT_{NFA}** : **NFA** $\times$ **NFA** $\rightarrow$ **NFA**
TO FIRST Proof₃, FIRST Proof₄
6. BY ExTable₁ **COMPARING** $N, A_1 \circ A_2$
7. APPLY **RegLangIFFNFA** $\Leftarrow$
TO mk-AND-proof Proof₅, Proof₆

New possible proof strategy!

Use **NFAs** Only

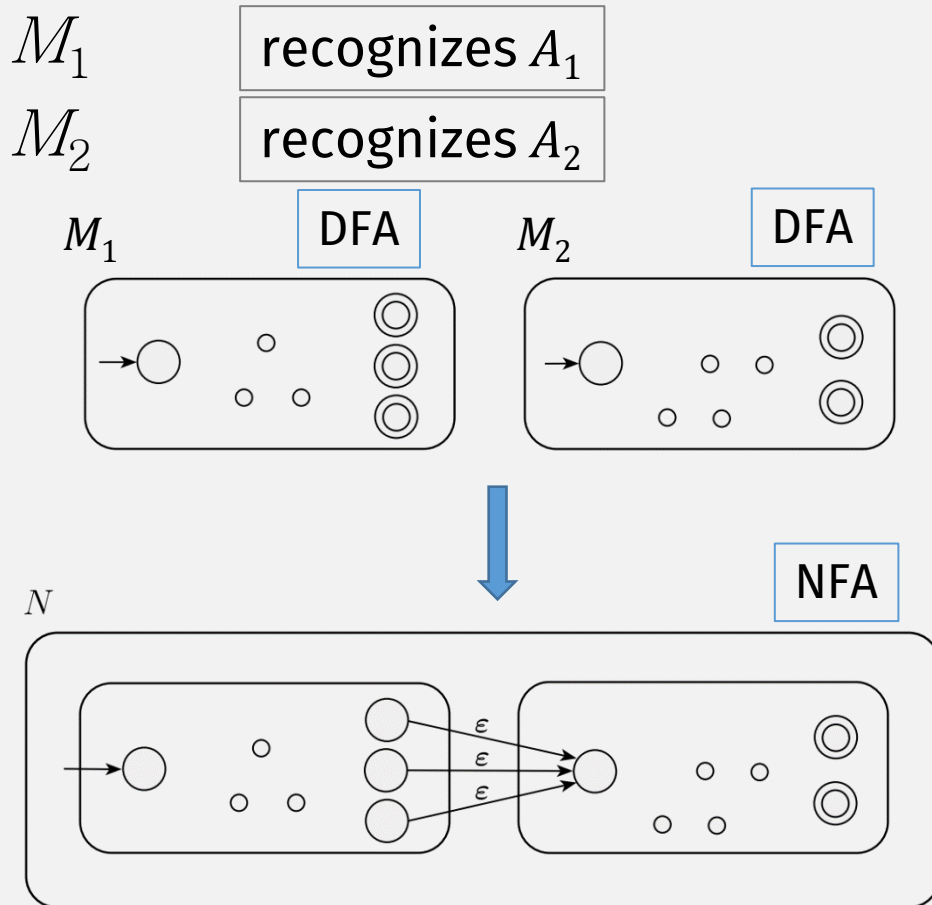
Define function ~~CONCAT~~_{DFA-NFA} : ~~DFA~~ × ~~DFA~~ → ~~NFA~~

~~CONCAT~~_{DFA-NFA} (~~M_1, M_2~~) = ~~N~~ , where

let ($Q_1, \Sigma, \delta_1, q_1, F_1$) = ~~get-DFA-parts~~ M_1

let ($Q_2, \Sigma, \delta_2, q_2, F_2$) = ~~get-DFA-parts~~ M_2

(Output) $N = \mathbf{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$ where



Use **NFAs** Only

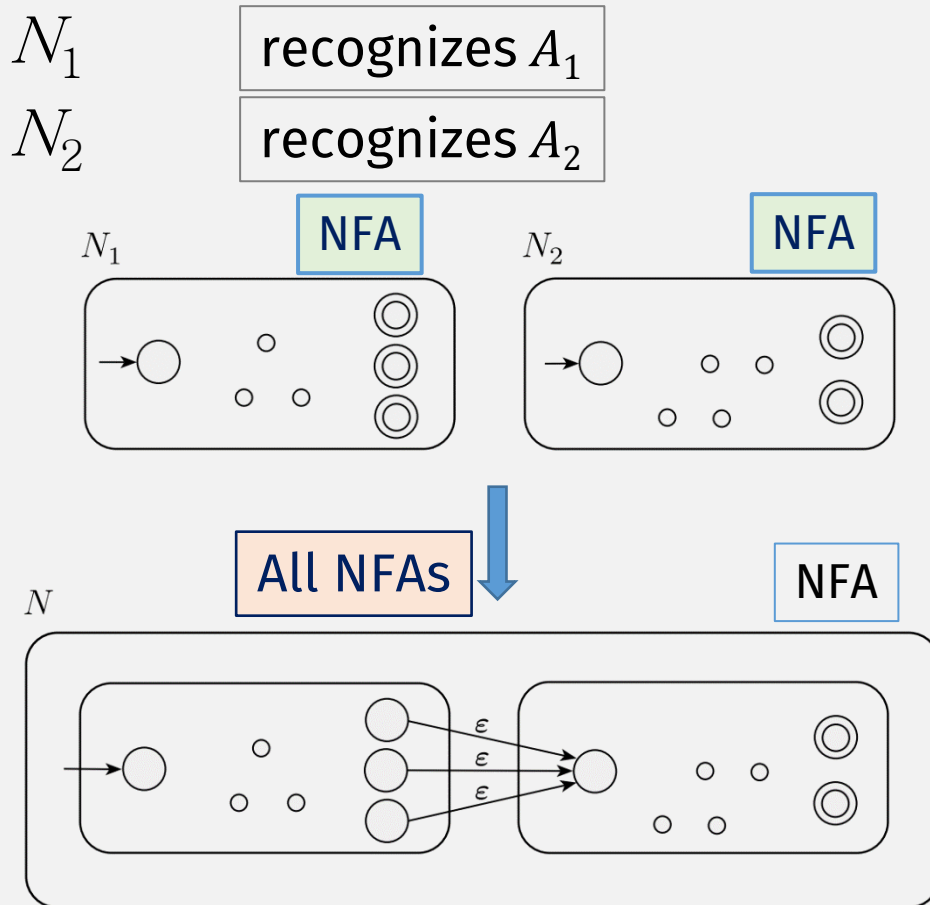
Define function $\text{CONCAT}_{\text{NFA}} : \text{NFA} \times \text{NFA} \rightarrow \text{NFA}$

$\text{CONCAT}_{\text{NFA}}(N_1, N_2) = N$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-NFA-parts } N_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-NFA-parts } N_2$

(Output) $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$ where



Use **NFAs** Only

Define function $\text{CONCAT}_{\text{NFA}} : \text{NFA} \times \text{NFA} \rightarrow \text{NFA}$

$\text{CONCAT}_{\text{NFA}}(N_1, N_2) = N$, where

let $(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-NFA-parts } N_1$

let $(Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-NFA-parts } N_2$

(Output) $N = \text{mk-NFA}(Q, \Sigma, \delta, q_1, F_2)$ where

$$Q = Q_1 \cup Q_2$$

The state q_1 is the same as the start state of N_1

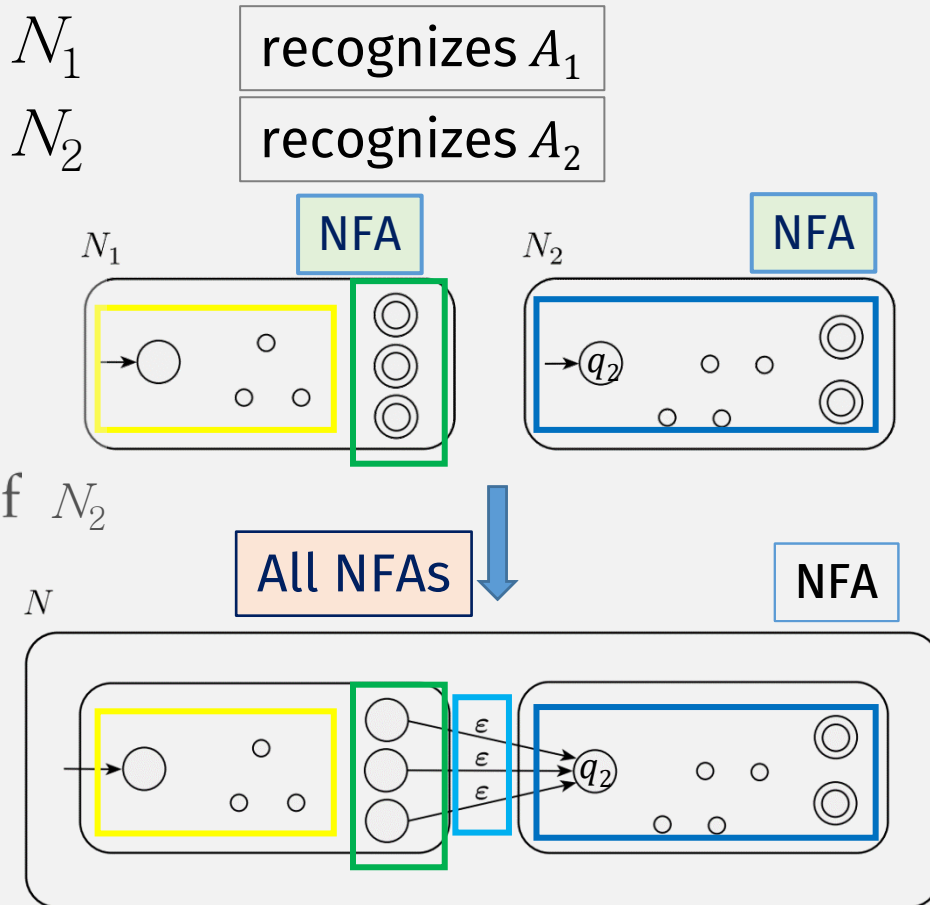
The accept states F_2 are the same as the accept states of N_2

$\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ where: for any $q \in Q$ and any $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ ? & q \in F_1 \text{ and } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

Already a set!
(why?)

Need to handle
existing epsilons!



Flashback!

Regular Languages Closed Under Union

THEOREM

The class of regular languages is closed under the union operation.

In other words, IF A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

Previously

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A **DFA** Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO FIRST Proof₃, FIRST PROOF₄
6. BY ExTable₁ COMPARING $M, A_1 \cup A_2$
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆ ■

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
 THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A ~~DFA~~ Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A ~~DFA~~ Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
 IS-A ~~DFA~~ Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. ~~APPLY DefRegLangCore~~
 TO Proof₁
4. ~~APPLY DefRegLangCore~~
 TO Proof₂
5. ~~APPLY UNION_{DFA} : DFA × DFA → DFA~~
 TO FIRST Proof₃, FIRST PROOF₄
6. BY ExTable₁ **COMPARING** $M, A_1 \cup A_2$
7. ~~APPLY DefRegLang~~
 TO mk-AND-proof Proof₅, Proof₆

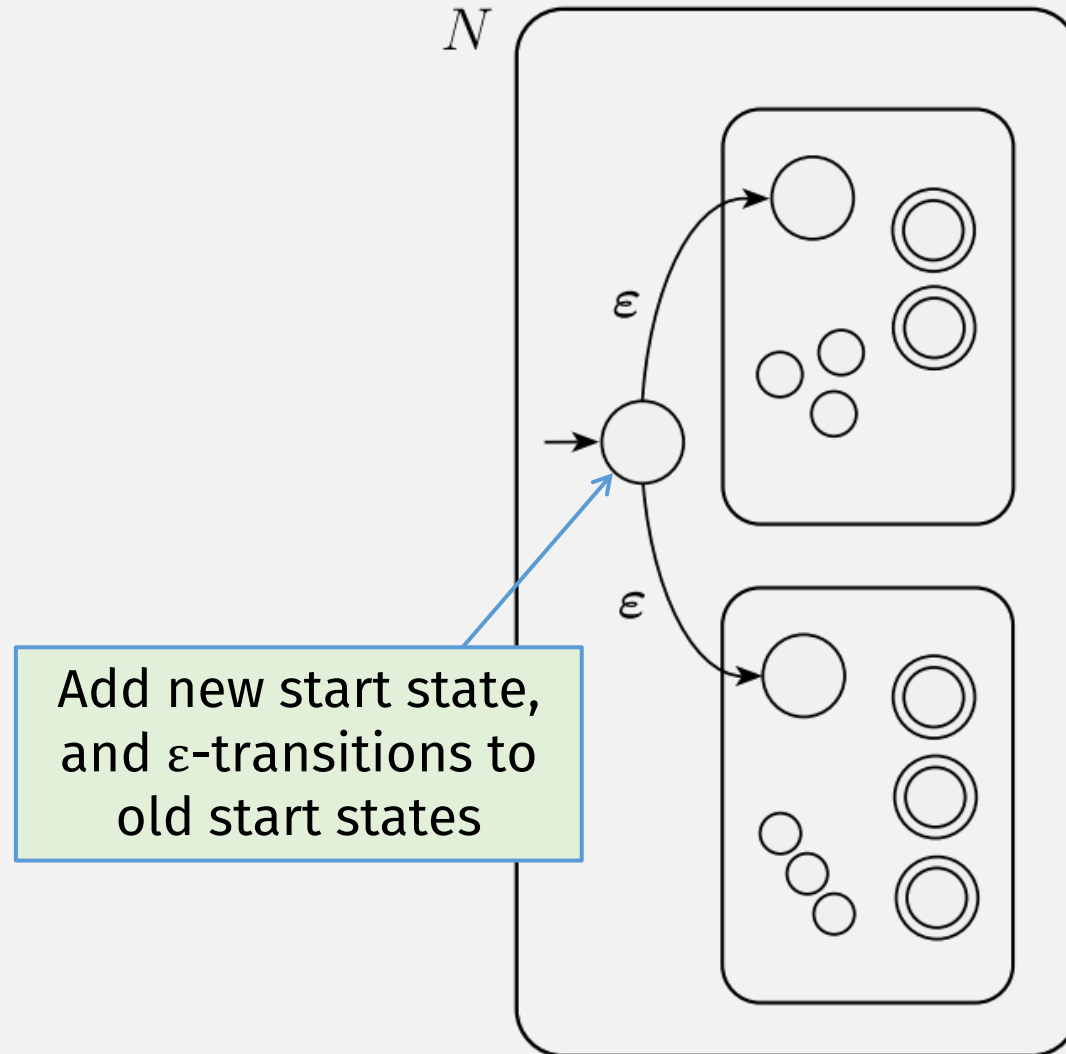
Are Regular Languages Closed Under Union

Alternate Function, on NFAs?

Define function $\text{UNION}_{\text{NFA}} : \text{NFA} \times \text{NFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{NFA}} (N_1, N_2) = N$, where ...

Union is Closed for Regular Languages



Define function $\text{UNION}_{\text{NFA}} : \text{NFA} \times \text{NFA} \rightarrow \text{NFA}$

$\text{UNION}_{\text{NFA}}(N_1, N_2) = N$, where

$\text{recognizes } A_1 (Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-NFA-parts } N_1$

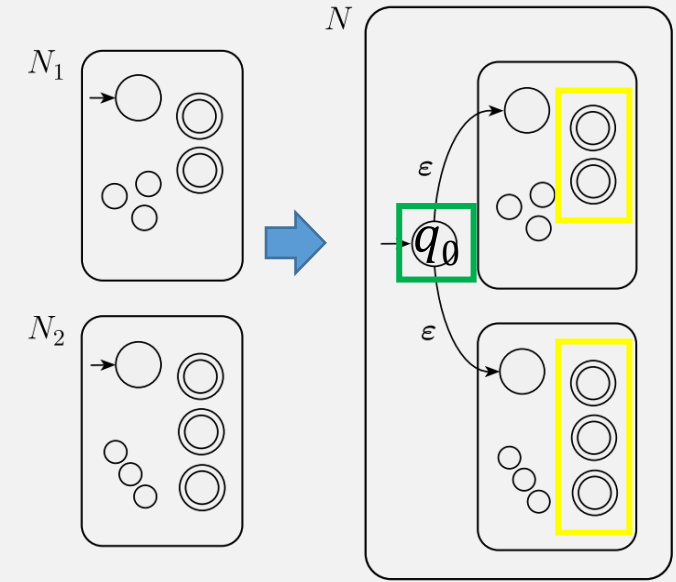
$\text{recognizes } A_2 (Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-NFA-parts } N_1$

$\text{recognizes } A_1 \cup A_2 N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F)$ where

$$Q = \{q_0\} \cup Q_1 \cup Q_2.$$

The state q_0 is ... a new start state!

The set of accept states $F = F_1 \cup F_2$.



Define function $\text{UNION}_{\text{NFA}} : \text{NFA} \times \text{NFA} \rightarrow \text{NFA}$

$\text{UNION}_{\text{NFA}}(N_1, N_2) = N$, where

$\text{recognizes } A_1 (Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-NFA-parts } N_1$

$\text{recognizes } A_2 (Q_2, \Sigma, \delta_2, q_2, F_2) = \text{get-NFA-parts } N_1$

$\text{recognizes } A_1 \cup A_2 N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F)$ where

$$Q = \{q_0\} \cup Q_1 \cup Q_2.$$

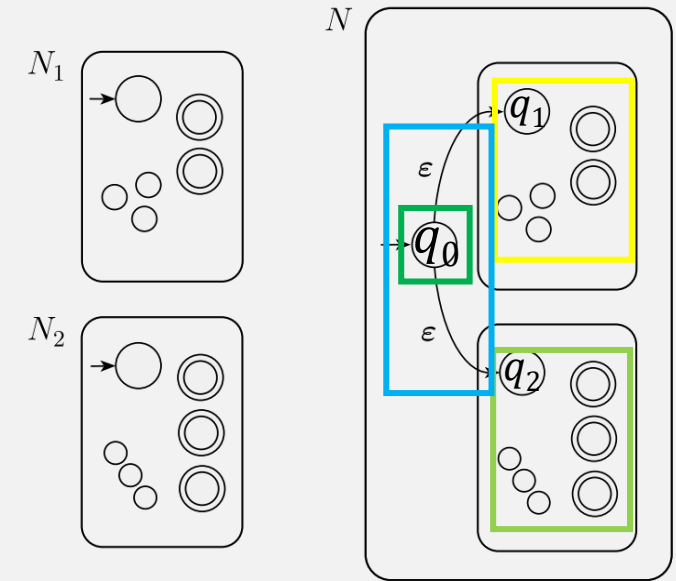
The state q_0 is ... a new start state!

The set of accept states $F = F_1 \cup F_2$.

$\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ where: for any $q \in Q$ and any $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(? , a) & q \in Q_1 \\ \delta_2(? , a) & q \in Q_2 \\ \{q_1 ? q_2\} & q = q_0 \text{ and } a = \epsilon \\ \emptyset & ? \end{cases}$$

New start state
should have only
epsilon
transitions!



List of Closed Ops for Reg Langs (so far)

☒ • Union

☒ • Concatentation

• Kleene Star (repetition) ?

Star: $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$

Kleene Star Example

Let the alphabet Σ be the standard 26 letters $\{a, b, \dots, z\}$.

If $A = \{\text{good}, \text{bad}\}$

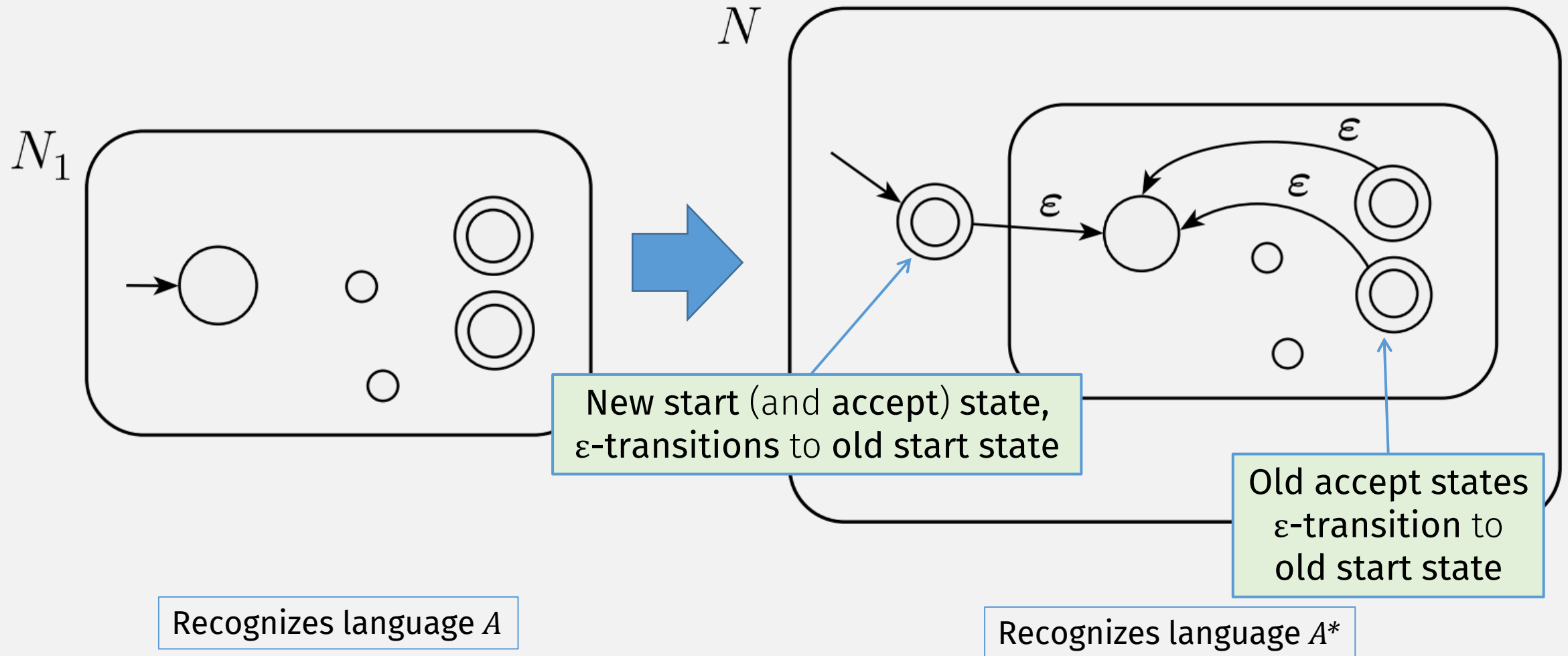
$A^* = \{\epsilon, \text{good}, \text{bad}, \text{goodgood}, \text{goodbad}, \text{badgood}, \text{badbad},$
 $\text{goodgoodgood}, \text{goodgoodbad}, \text{goodbadgood}, \text{goodbadbad}, \dots\}$

Note: repeat zero or more times

(this is an infinite language!)

Star: $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$

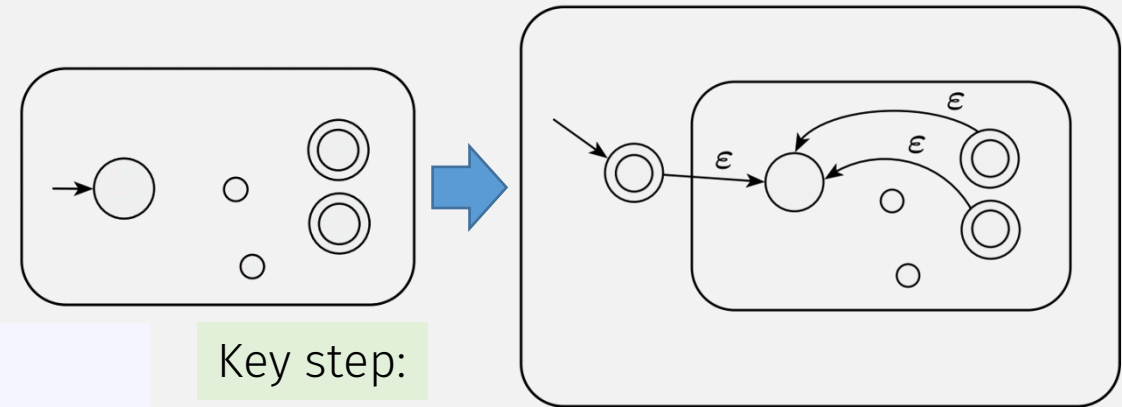
Kleene Star is Closed for Regular Langs?



Kleene Star is Closed for Regular Langs

THEOREM

The class of regular languages is closed under the star operation.



Key step:

$\text{STAR}_{\text{NFA}} : \text{NFA} \rightarrow \text{NFA}$

where $\text{LANGOF}(\text{STAR}_{\text{NFA}}(N)) = L(N)^*$

Define function $\text{STAR}_{\text{NFA}} : \text{NFA} \rightarrow \text{NFA}$

$\text{STAR}_{\text{NFA}}(N_1) = N$, where

$(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-NFA-parts } N_1$

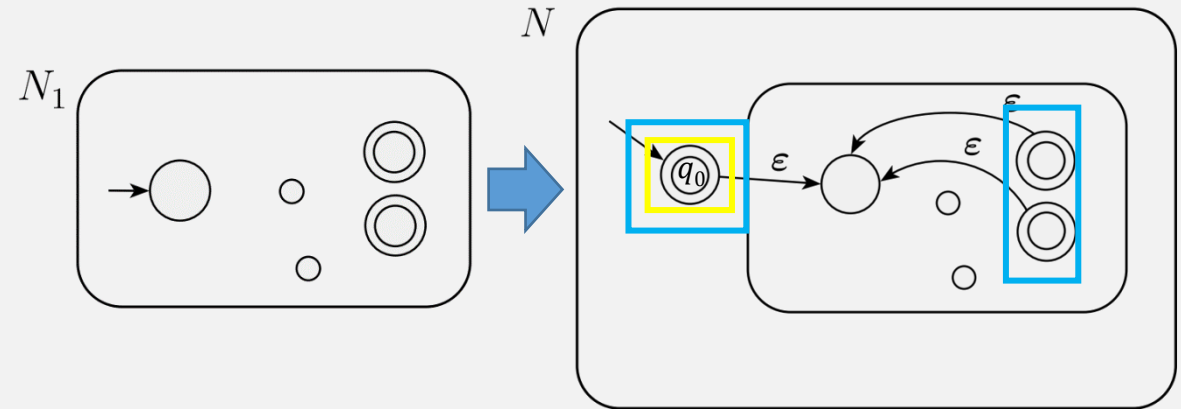
$N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F)$ where

$$Q = \{q_0\} \cup Q_1$$

The state q_0 is ... a new start state!

$$F = \{q_0\} \cup F_1$$

Kleene star of a language must accept the empty string!



Define function $\text{STAR}_{\text{NFA}} : \text{NFA} \rightarrow \text{NFA}$

$\text{STAR}_{\text{NFA}}(N_1) = N$, where

$(Q_1, \Sigma, \delta_1, q_1, F_1) = \text{get-NFA-parts } N_1$

$N = \text{mk-NFA}(Q, \Sigma, \delta, q_0, F)$ where

$$Q = \{q_0\} \cup Q_1$$

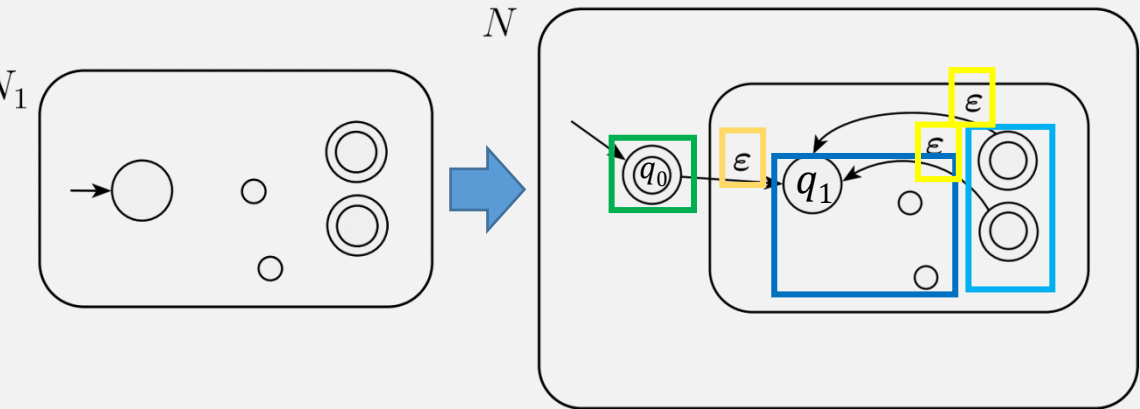
The state q_0 is ... a new start state!

$$F = \{q_0\} \cup F_1$$

$\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ where: for any $q \in Q$ and any $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \epsilon \\ \{q_1\} & q = q_0 \text{ and } a = \epsilon \\ \emptyset & q = q_0 \text{ and } a \neq \epsilon. \end{cases}$$

F_1 states might already have empty transitions!



Old accept states ϵ -transition to old start state

New start state ϵ -transitions to old start state

New start state has only ϵ -transitions

Why These (Closed) Operations?

- Union

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

- Concatenation

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$

- Kleene star (repetition)

$$A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$$

All regular languages can be constructed from:

- (language of) **single-char strings** (from some alphabet), and
- these **three closed operations**!